

数学实验与数学软件

第三次：补充课实验

谭军

中山大学数学学院

教师邮箱：mcstj@mail.sysu.edu.cn



PCA主成分分析

principal component analysis

目录 Contents

01 PCA的背景分析

02 PCA推导

03 PCA的算法流程

04 实例分析

05 PCA应用到地面分割

01 PCA的背景分析

PCA (Principal Component Analysis) 是一种常用的数据分析方法。PCA通过线性变换将原始数据变换为一组各维度线性无关的表示，可用于提取数据的主要特征分量，常用于高维数据的降维。PCA的优点是简单，而且无参数限制，可以方便的应用与各个场合。

01 PCA的背景分析

一般情况下，在数据挖掘和机器学习中，数据被表示为向量。例如某个淘宝店2012年全年的流量及交易情况可以看成一组记录的集合，其中每一天的数据是一条记录，格式如下：

(日期, 浏览量, 访客数, 下单数, 成交数, 成交金额)

每条记录可以被表示为一个五维向量，其中一条看起来大约是这个样子：

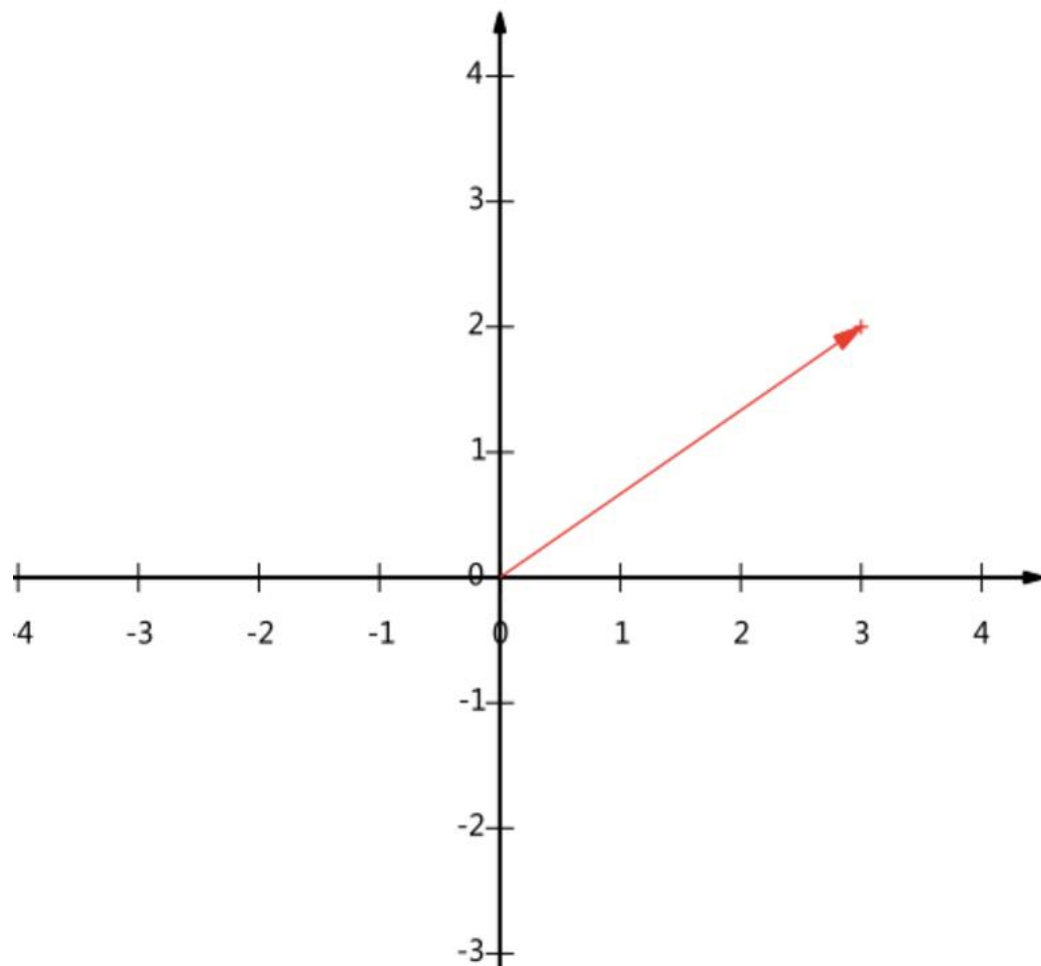
$$(500, 240, 25, 13, 2312.15)^T$$

01 PCA的背景分析

机器学习算法的复杂度和数据的维数有着密切关系，甚至与维数呈指数级关联，在这种情况下，机器学习的资源消耗是不可接受的，因此我们必须对数据进行降维。降维当然意味着信息的丢失，不过鉴于实际数据本身常常存在的相关性，我们可以想办法在降维的同时将信息的损失尽量降低。

降维的朴素思想描述：**如果我们删除数据中的一个指标，并不会丢失太多原本信息。因此我们可以删除一个，以降低机器学习算法的复杂度。**

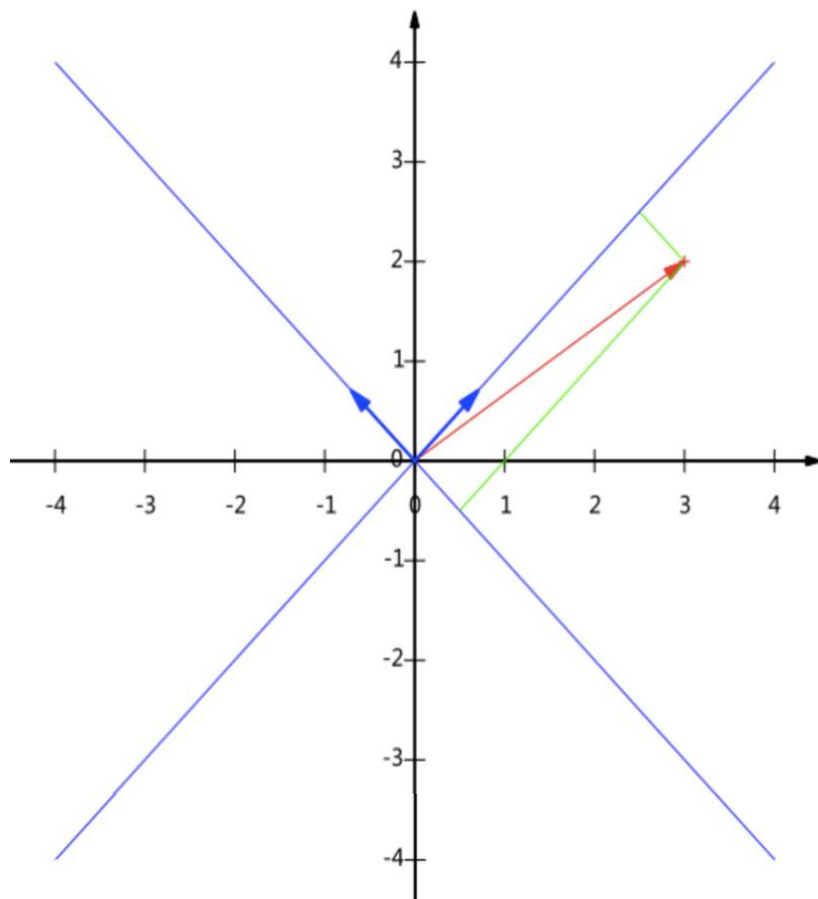
02 PCA推导



图中的向量可以表示为 $(3,2)$ 。而我们常常忽略只有一个 $(3,2)$ 本身是不能够精确表示一个向量的。这里的3实际表示的是向量在x轴上的投影值是3，在y轴上的投影值是2。

更正式的说，向量 (x,y) 实际上表示线性组合： $x(1,0)^T + y(0,1)^T$

02 PCA推导



所以，要准确描述向量，首先要确定一组基，然后给出在基所在的各个直线上的投影值，就可以了。

我们也可以选择其他的一对正交的向量作为基来描述其他的向量。例如我们使用 $(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}})^T$ 和 $(\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}})^T$ 。

此时向量 $(3, 2)$ 在该基下的新坐标为 $(\frac{5}{\sqrt{2}}, -\frac{1}{\sqrt{2}})$

02 PCA推导

向量(3,2)可以通过下面的矩阵运算来获得新的坐标

$$\begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix} \begin{pmatrix} 3 \\ 2 \end{pmatrix} = \begin{pmatrix} \frac{5}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{pmatrix}$$

一般的，如果我们有M个N维向量，想将其变换为由R个N维向量表示的新空间中，那么首先将R个基按行组成矩阵A，然后将向量按列组成矩阵B，那么两矩阵的乘积AB就是变换结果，其中AB的第m列为A中第m列变换后的结果

$$\begin{pmatrix} p_1 \\ p_2 \\ \vdots \\ p_R \end{pmatrix} (a_1 \quad a_2 \quad \cdots \quad a_M) = \begin{pmatrix} p_1 a_1 & p_1 a_2 & \cdots & p_1 a_M \\ p_2 a_1 & p_2 a_2 & \cdots & p_2 a_M \\ \vdots & \vdots & \ddots & \vdots \\ p_R a_1 & p_R a_2 & \cdots & p_R a_M \end{pmatrix}$$

02 PCA推导

那么问题来了：如何选择基才是最优的？或者说，如果我们有一组N维向量，现在要将其降到K维（K小于N），那么我们应该如何选择K个基才能最大程度保留原有的信息？

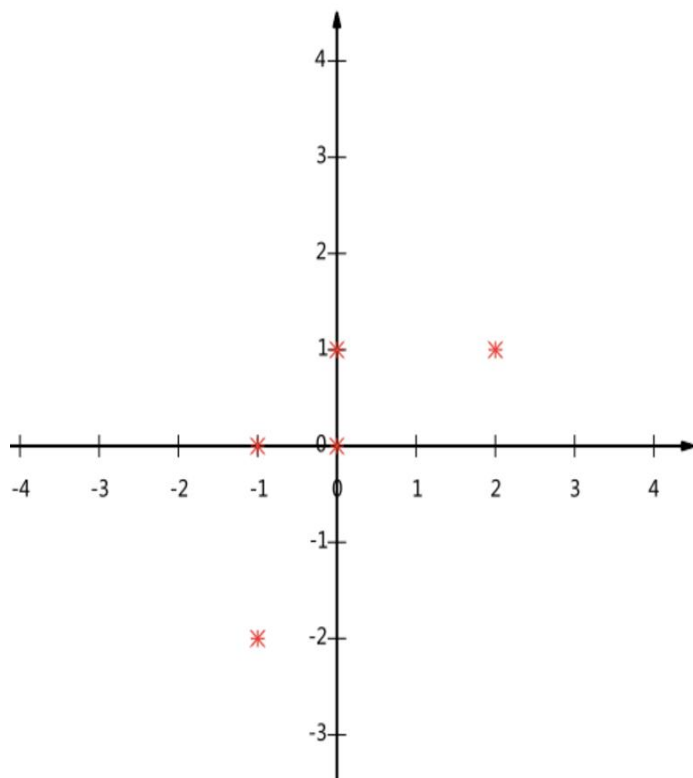
例：假设我们的数据由五条记录组成，将它们表示成矩阵形式：

$$\begin{pmatrix} 1 & 1 & 2 & 4 & 2 \\ 1 & 3 & 3 & 4 & 4 \end{pmatrix}$$

减去每个字段的均值

$$\begin{pmatrix} -1 & -1 & 0 & 2 & 0 \\ -2 & 0 & 0 & 1 & 1 \end{pmatrix}$$

02 PCA推导



如果我们必须使用一维来表示这些数据，又希望尽量保留原始的信息，应如何选择？

一种直观的看法是：**希望投影后的投影值尽可能分散**。这种分散程度，可以用数学上的方差来表述：

$$Var(a) = \frac{1}{m} \sum_{i=1}^m (a_i - \mu)^2$$

02 PCA推导

由于我们已经将每个字段的均值都化为0了，因此方差可以直接用每个元素的平方和除以元素个数表示：

$$Var(a) = \frac{1}{m} \sum_{i=1}^m a_i^2$$

于是上面的问题被形式化表述为：寻找一个一维基，使得所有数据变换为这个基上的坐标表示后，方差值最大。

02 PCA推导

数学上可以用两个字段的协方差表示其相关性，由于已经让每个字段均值为0，则：

$$Cov(a, b) = \frac{1}{m} \sum_{i=1}^m a_i b_i$$

当协方差为0时，表示两个字段完全独立。为了让协方差为0，我们选择第二个基时只能在与第一个基正交的方向上选择。因此最终选择的两个方向一定是正交的。

02 PCA推导

至此，我们得到了降维问题的优化目标：将一组N维向量降为K维（K大于0，小于N），其目标是选择K个单位（模为1）正交基，使得原始数据变换到这组基上后，各字段两两间协方差为0，而字段的方差则尽可能大（在正交的约束下，取最大的K个方差）。

假设我们只有a和b两个字段，那么我们将它们按行组成矩阵X：

$$X = \begin{pmatrix} a_1 & a_2 & \cdots & a_m \\ b_1 & b_2 & \cdots & b_m \end{pmatrix}$$

然后我们用X乘以X的转置，并乘上系数1/m：

$$\frac{1}{m}XX^T = \begin{pmatrix} \frac{1}{m} \sum_{i=1}^m a_i^2 & \frac{1}{m} \sum_{i=1}^m a_i b_i \\ \frac{1}{m} \sum_{i=1}^m a_i b_i & \frac{1}{m} \sum_{i=1}^m b_i^2 \end{pmatrix}$$

02 PCA推导

推广：

设我们有 m 个 n 维数据记录，将其按列排成 n 乘 m 的矩阵 X ，设 $C = \frac{1}{m}XX^T$ ，则 C 是一个对称矩阵，其对角线分别个各个字段的方差，而第 i 行 j 列和 j 行 i 列元素相同，表示 i 和 j 两个字段的协方差

02 PCA推导

假设原始数据矩阵 X 对应的协方差矩阵为 C ，而 P 是一组基按行组成的矩阵，设 $Y=PX$ ，则 Y 为 X 对 P 做基变换后的数据。设 Y 的协方差矩阵为 D ，我们推导一下 D 与 C 的关系：

$$\begin{aligned} D &= \frac{1}{m} Y Y^T \\ &= \frac{1}{m} (P X) (P X)^T \\ &= \frac{1}{m} P X X^T P^T \\ &= P \left(\frac{1}{m} X X^T \right) P^T \\ &= P C P^T \end{aligned}$$

优化目标变成了寻找一个矩阵 P ，满足 $P C P^T$ 是一个对角矩阵，并且对角元素按从大到小依次排列，那么 P 的前 K 行就是要寻找的基，用 P 的前 K 行组成的矩阵乘以 X 就使得 X 从 N 维降到了 K 维并满足上述优化条件。

02 PCA推导

实对称矩阵的两个性质：

1) 实对称矩阵不同特征值对应的特征向量必然正交。

2) 设特征值 λ 重数为 r ，则必然存在 r 个线性无关的特征向量对应于 λ ，因此可以将这 r 个特征向量单位正交化。

对协方差矩阵 C 有如下结论：

$$E^T C E = \Lambda = \begin{pmatrix} \lambda_1 & & & \\ & \lambda_2 & & \\ & & \ddots & \\ & & & \lambda_n \end{pmatrix}$$

其中 $E = (e_1, e_2, \dots, e_n)$ 。

因此我们前面需要的矩阵 $P = E^T$

03 PCA的算法流程

总结一下PCA的算法步骤：

设有 m 条 n 维数据。

- 1) 将原始数据按列组成 n 行 m 列矩阵 X
- 2) 将 X 的每一行（代表一个属性字段）进行零均值化，即减去这一行的均值
- 3) 求出协方差矩阵 $C = \frac{1}{m}XX^T$
- 4) 求出协方差矩阵的特征值及对应的特征向量
- 5) 将特征向量按对应特征值大小从上到下按行排列成矩阵，取前 k 行组成矩阵 P
- 6) $Y = PX$ 即为降维到 k 维后的数据

04 实例分析

我们以之前的数据为例，用PCA方法将这组二维数据其降到一维。

$$C = \frac{1}{5} \begin{pmatrix} -1 & -1 & 0 & 2 & 0 \\ -2 & 0 & 0 & 1 & 1 \end{pmatrix} \begin{pmatrix} -1 & -2 \\ -1 & 0 \\ 0 & 0 \\ 2 & 1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} \frac{6}{5} & \frac{4}{5} \\ \frac{4}{5} & \frac{6}{5} \end{pmatrix}$$

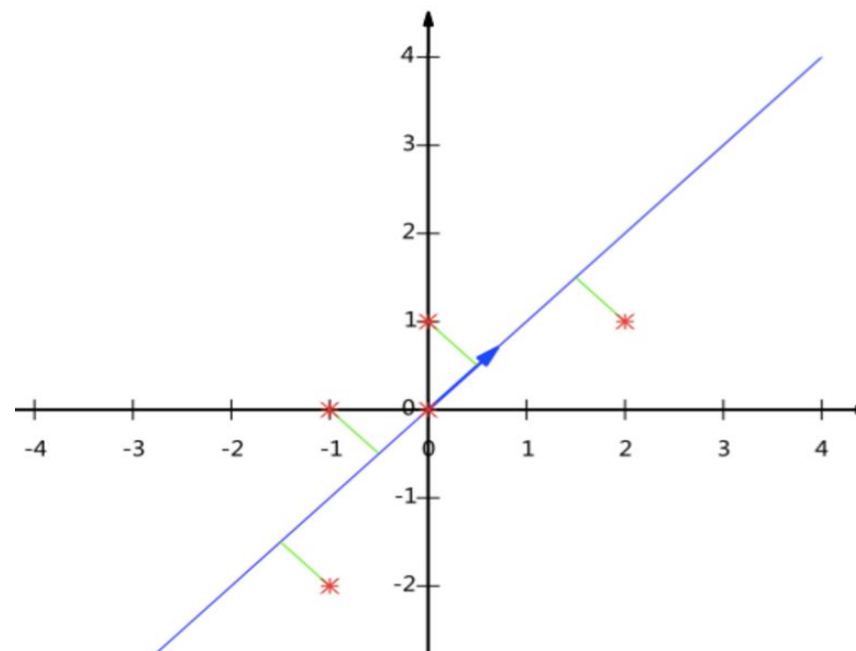
计算该协方差矩阵的特征值和特征向量：

$$\lambda_1 = 2, \lambda_2 = 2/5 \quad c_1 \begin{pmatrix} 1 \\ 1 \end{pmatrix}, c_2 \begin{pmatrix} -1 \\ 1 \end{pmatrix}$$

04 实例分析

我们将特征向量标准化:

$$P = \begin{pmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ -1/\sqrt{2} & 1/\sqrt{2} \end{pmatrix}$$



最后我们用P的第一行乘以数据矩阵，就得到了降维后的表示:

$$Y = (1/\sqrt{2} \quad 1/\sqrt{2}) \begin{pmatrix} -1 & -1 & 0 & 2 & 0 \\ -2 & 0 & 0 & 1 & 1 \end{pmatrix} = (-3/\sqrt{2} \quad -1/\sqrt{2} \quad 0 \quad 3/\sqrt{2} \quad -1/\sqrt{2})$$

MATLAB操作

```
X=[-1,-1,0,2,0;-2,0,0,1,1]
```

```
X = [ -1  -1   0   2   0  
      -2   0   0   1   1]
```

```
[c,s,l]=pca(X')
```

[coeff,score,latent] = pca(____) 还在 score 中返回主成分分数，在 latent 中返回主成分方差。您可以使用上述语法中的任何输入参数。

主成分分数是 X 在主成分空间中的表示。
score 的行对应于观测值，列对应于成分。
主成分方差是 X 的协方差矩阵的特征值。

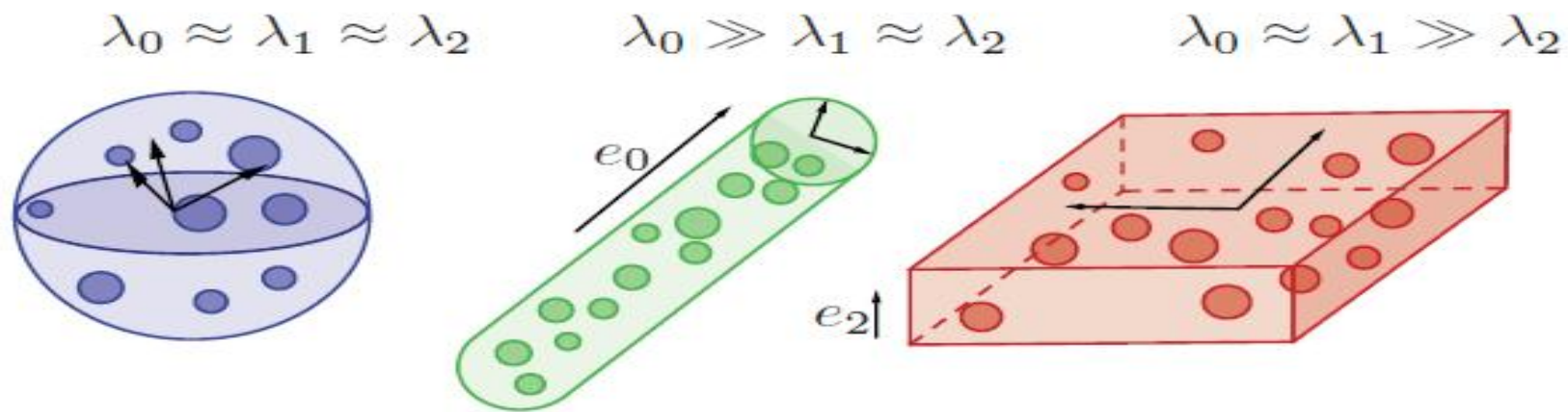
```
c =  0.7071  0.7071  
     0.7071 -0.7071
```

```
s = -2.1213  0.7071  
     -0.7071 -0.7071  
         0      0  
     2.1213  0.7071  
     0.7071 -0.7071
```

```
l =  2.5000  
     0.5000
```

05 PCA应用到地面分割

- 1) 如果三个特征值近似相同，则当前的栅格中的坐标是没有主方向的，是非结构化环境。
- 2) 如果其中一个特征值远大于另外两个特征值，那么点云形状为柱状，大的特征值为该柱状点云的主方向
- 3) 如果其中一个特征值远小于另外两个特征值，则该点云为平面点云。



K-means 聚类算法

k -means算法，也被称为 k -平均或 k -均值，是一种得到最广泛使用的聚类算法。它是将各个聚类子集内的所有数据样本的均值作为该聚类的代表点，算法的主要思想是通过迭代过程把数据集划分为不同的类别，使得评价聚类性能的准则函数达到最优，从而使生成的每个聚类内紧凑，类间独立。这一算法不适合处理离散型属性，但是对于连续型具有较好的聚类效果。

划分聚类方法对数据集进行聚类时包括如下

三个要点：

- **(1) 选定某种距离作为数据样本间的相似性度量**

上面讲到，k-means聚类算法不适合处理离散型属性，对连续型属性比较适合。因此在计算数据样本之间的距离时，可以根据实际需要选择欧式距离、曼哈顿距离或者明考斯距离中的一种来作为算法的相似性度量，其中最常用的是欧式距离。下面具体介绍一下欧式距离。

假设给定的数据集 $X = \{x_m \mid m = 1, 2, \dots, total\}$ ，X中的样本用d个描述属性 $A_1, A_2, \dots, A_d$ 来表示，并且d个描述属性都是连续型属性。

数据样本 $x_i = (x_{i1}, x_{i2}, \dots, x_{id})$ ， $x_j = (x_{j1}, x_{j2}, \dots, x_{jd})$ 其中， $x_{i1}, x_{i2}, \dots, x_{id}$ 和 $x_{j1}, x_{j2}, \dots, x_{jd}$ 分别是样本 x_i 和 x_j 对应d个描述属性 $A_1, A_2, \dots, A_d$ 的具体取值。

样本 x_i 和 x_j 之间的相似度通常用它们之间的距离 $d(x_i, x_j)$ 来表示，

- 距离越小，样本 x_i 和 x_j 越相似，差异度越小；
- 距离越大，样本 x_i 和 x_j 越不相似，差异度越大。

欧式距离公式如下：

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^d (x_{ik} - x_{jk})^2}$$

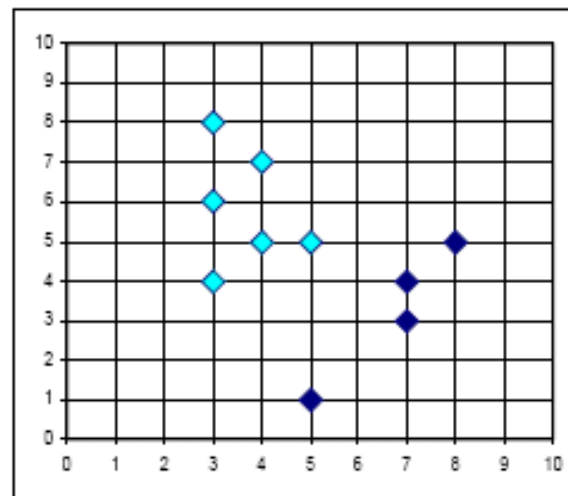
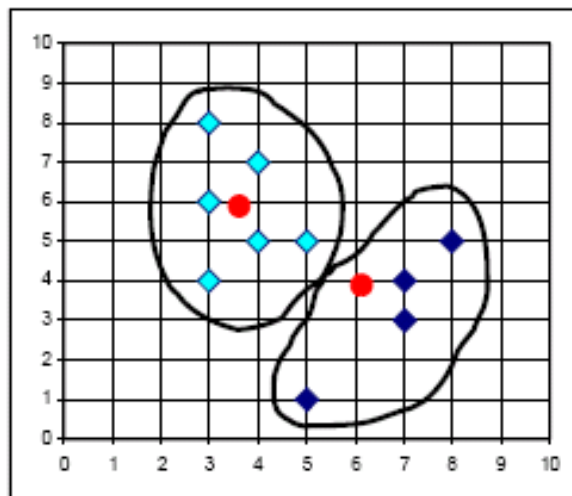
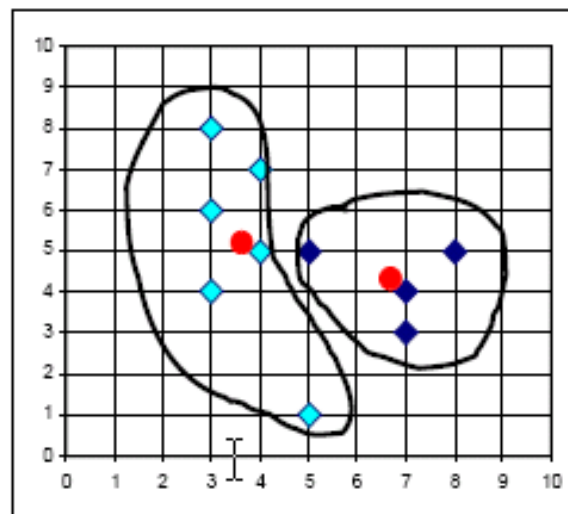
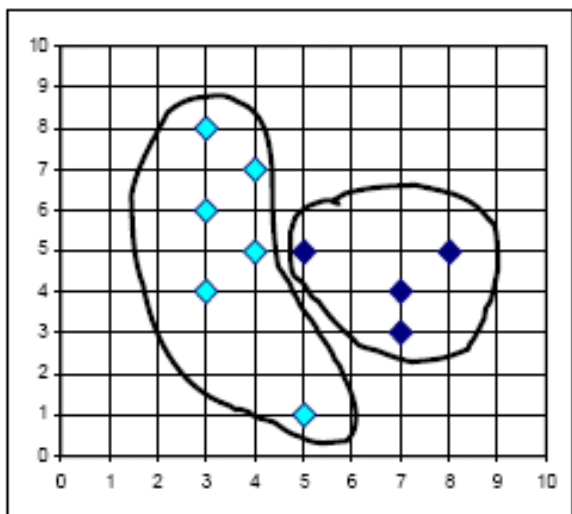
- **(2) 选择评价聚类性能的准则函数**

k-means聚类算法使用误差平方和准则函数来评价聚类性能。给定数据集 X ，其中只包含描述属性，不包含类别属性。假设 X 包含 k 个聚类子集 $X_1, X_2, \dots, X_K$ ；各个聚类子集中的样本数量分别为 $n_1, n_2, \dots, n_k$ ；各个聚类子集的均值代表点（也称聚类中心）分别为 $m_1, m_2, \dots, m_k$ 。则误差平方和准则函数公式为：

$$E = \sum_{i=1}^k \sum_{p \in X_i} \|p - m_i\|^2$$

(3) 相似度的计算根据一个簇中对象的平均值来进行。

- 1) 将所有对象随机分配到 k 个非空的簇中。
- 2) 计算每个簇的平均值，并用该平均值代表相应的簇。
- 3) 根据每个对象与各个簇中心的距离，分配给最近的簇。
- 4) 然后转 (2)，重新计算每个簇的平均值。这个过程不断重复直到满足某个准则函数才停止。



K-均值聚类示例

K-means算法2个核心问题:

- 1.度量记录之间的相关性的计算公式, 此处采用欧式距离。
- 2.更新簇内质心的方法, 此处采用平均值法, 即means。

输入: 簇的数目 k 和包含 n 个对象的数据库。

输出: k 个簇, 使平方误差准则最小。

方法: 基于簇中对象的平均值。

(1) 任意选择 k 个对象作为初始的簇中心;

(2) repeat,

(3) 根据簇中对象的平均值, 将每个对象(重新) 赋给最类似的簇:

(4) 更新簇的平均值, 即计算每个簇中对象的平均值;

(5) until 不再发生变化。

算法 *k*-means算法

输入：簇的数目 k 和包含 n 个对象的数据库。

输出： k 个簇，使平方误差准则最小。

(1) assign initial value for means; /*任意分配到 k 个对象作为簇的平均值*/

(2) REPEAT

(3) FOR $j=1$ to n DO assign each x_j to the closest clusters;

(4) FOR $i=1$ to k DO $\bar{x}_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$ /*更新簇平均值*/

(5) Compute $E = \sum_{i=1}^k \sum_{x \in C_i} |x - \bar{x}_i|^2$ /*计算准则函数 E */

(6) UNTIL E 不再明显地发生变化。

例子

O	x	y
1	0	2
2	0	0
3	1.5	0
4	5	0
5	5	2

数据对象集合S见表1，作为一个聚类分析的二维样本，要求的簇的数量 $k=2$ 。

(1) 选择 $O_1(0,2)$, $O_2(0,0)$ 为初始的簇中心，
即 $M_1 = O_1 = (0,2)$, $M_2 = O_2 = (0,0)$

(2) 对剩余的每个对象，根据其与各个簇中心的距离，将它赋给最近的簇。

对 O_3 :

$$d(M_1, O_3) = \sqrt{(0-1.5)^2 + (2-0)^2} = 2.5$$

$$d(M_2, O_3) = \sqrt{(0-1.5)^2 + (0-0)^2} = 1.5$$

显然 $d(M_2, O_3) \leq d(M_1, O_3)$ ，故将 O_3 分配给 C_2 ；同理，将 O_4 分配给 C_2 ， O_5 分配给 C_1 。

更新，得到新簇 $C_1 = \{O_1, O_5\}$ 和 $C_2 = \{O_2, O_3, O_4\}$

计算平方误差准则，单个方差为

$$E_1 = [(0-0)^2 + (2-2)^2] + [(0-5)^2 + (2-2)^2] = 25 \quad E_2 = 27.25$$

总体平均方差是： $E = E_1 + E_2 = 25 + 27.25 = 52.25$

(3) 计算新的簇的中心。

$$M_1 = ((0+5)/2, (2+2)/2) = (2.5, 2)$$

$$M_2 = ((0+1.5+5)/3, (0+0+0)/3) = (2.17, 0)$$

重复 (2) 和 (3)，得到 O_1 分配给 C_1 ； O_2 分配给 C_2 ， O_3 分配给 C_2 ， O_4 分配给 C_2 ， O_5 分配给 C_1 。更新，得到新簇 $C_1 = \{O_1, O_5\}$

和 $C_2 = \{O_2, O_3, O_4\}$ 。中心为 $M_1 = (2.5, 2)$ ， $M_2 = (2.17, 0)$ 。显影的单个方差分别为

$$E_1 = [(0-2.5)^2 + (2-2)^2] + [(2.5-5)^2 + (2-2)^2] = 12.5 \quad E_2 = 13.15$$

总体平均误差是： $E = E_1 + E_2 = 12.5 + 13.15 = 25.65$

由上可以看出，第一次迭代后，总体平均误差值52.25~25.65，显著减小。由于在两次迭代中，簇中心不变，所以停止迭代过程，算法停止。

***k*-means算法的性能分析**

■主要优点:

- ◆是解决聚类问题的一种经典算法，简单、快速。
- ◆对处理大数据集，该算法是相对可伸缩和高效率的。因为它的复杂度是 $O(n k t)$ ，其中， n 是所有对象的数目， k 是簇的数目， t 是迭代的次数。通常 $k \ll n$ 且 $t \ll n$ 。
- ◆当结果簇是密集的，而簇与簇之间区别明显时，它的效果较好。

■主要缺点

- ◆在簇的平均值被定义的情况下才能使用，这对于处理符号属性的数据不适用。
- ◆必须事先给出 k （要生成的簇的数目），而且对初值敏感，对于不同的初始值，可能会导致不同结果。
- ◆它对于“噪声”和孤立点数据是敏感的，少量的该类数据能够对平均值产生极大的影响。

k-means算法的改进方法——k-mode 算法

◆k-modes 算法：实现对离散数据的快速聚类，保留了k-means算法的效率同时将k-means的应用范围扩大到离散数据。

◆K-modes算法是按照k-means算法的核心内容进行修改，针对分类属性的度量和更新质心的问题而改进。

具体如下：

- 1.度量记录之间的相关性D的计算公式是比较两记录之间，属性相同为0，不同为1.并所有相加。因此D越大，即他的不相关程度越强（与欧式距离代表的意义是一样的）；
- 2.更新modes，使用一个簇的每个属性出现频率最大的那个属性值作为代表簇的属性值。

k-means算法的改进方法k-prototype算法

◆k-Prototype算法：可以对离散与数值属性两种混合的数据进行聚类，在k-prototype中定义了一个对数值与离散属性都计算的相异性度量标准。

K-Prototype算法是结合K-Means与K-modes算法，针对混合属性的，解决2个核心问题如下：

- 1.度量具有混合属性的方法是，数值属性采用K-means方法得到 $P1$ ，分类属性采用K-modes方法 $P2$ ，那么 $D=P1+a*P2$ ， a 是权重，如果觉得分类属性重要，则增加 a ，否则减少 a ， $a=0$ 时即只有数值属性
- 2.更新一个簇的中心的方法，方法是结合K-Means与K-modes的更新方法。

k-means算法的改进方法—k-中心点算法

k -中心点算法：

k -means算法对于孤立点是敏感的。为了解决这个问题，不采用簇中的平均值作为参照点，可以选用簇中位置最中心的对象，即中心点作为参照点。这样划分方法仍然是基于最小化所有对象与其参照点之间的相异度之和的原则来执行的。

k-Means matlab语法

`idx = kmeans(X,k)`

`idx = kmeans(X,k,Name,Value)`

`[idx,C] = kmeans(____)`

`[idx,C,sumd] = kmeans(____)`

`[idx,C,sumd,D] = kmeans(____)`

1, `idx = kmeans(X,k)` 执行 k 均值聚类，以将 $n \times p$ 数据矩阵 X 的观测值划分为 k 个聚类，并返回包含每个观测值的簇索引的 $n \times 1$ 向量 (`idx`)。X 的行对应于点，列对应于变量。

默认情况下，`kmeans` 使用欧几里德距离平方度量，并用 `k-means++` 算法进行簇中心初始化。

2, `[idx,C]` = `kmeans(____)` 在 $k \times p$ 矩阵 C 中返回 k 个簇质心的位置。

最小圆覆盖算法

最小圆覆盖

- 是数学中的一个算法问题，研究如何寻找能够覆盖平面上一群点的最小圆。这个问题在一般的 n 维空间中的推广是最小包围球的问题，即寻找能覆盖 n 维空间中某个点集的最小球。最小圆覆盖问题最早由十九世纪的英国数学家詹姆斯·约瑟夫·西尔维斯特在1857年提出。

问题描述

给定 n 个点，用一个最小的圆把这些点全部覆盖，求这个圆的圆心半径

一、核心原理：圆的确定条件

平面上的最小覆盖圆仅由两种情况决定，不存在“由 4 个及以上点同时在圆上”的可能（否则可找到更小的圆覆盖这些点）：

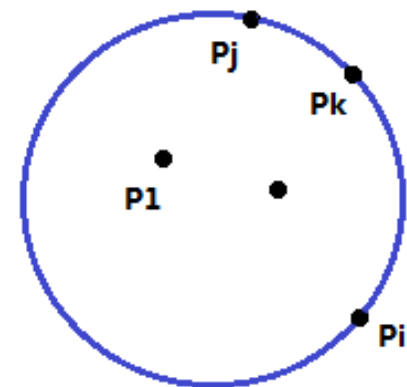
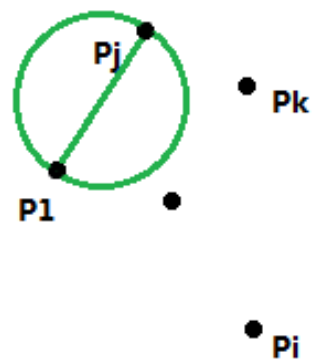
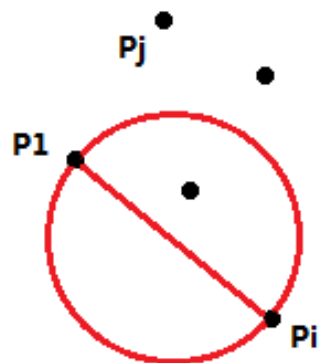
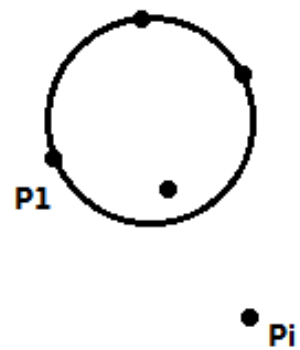
- 1) 1 个点确定：所有点重合，圆以该点为圆心，半径为 0。
- 2) 2 个点确定：圆以这两点的连线为直径（此时其他点均在圆内或圆上）。
- 3) 3 个点确定：这三点构成锐角或直角三角形，圆为该三角形的外接圆；若为钝角三角形，圆仍以最长边为直径（钝角顶点在圆内）。

二、经典算法：增量法 (Incremental Algorithm)

最常用的实现方式是“增量法”，通过逐步加入点并调整圆的范围，确保每一步的圆都覆盖已加入的所有点，具体步骤如下：

1. 算法流程 (3 层循环迭代)

- 外层循环：依次将每个点 P_i 作为“候选边界点”，若 P_i 在当前圆外，则以 P_i 为圆心、半径 0 重新初始化圆（此时圆仅覆盖 P_i ）。
- 中层循环：针对当前圆心 P_i ，依次将前 $(i-1)$ 个点 P_j 作为“候选边界点”，若 P_j 在当前圆外，则以 (P_i, P_j) 为直径重新初始化圆（此时圆覆盖 (P_i, P_j) ）。
- 内层循环：针对当前直径 (P_i, P_j) ，依次将前 $(j-1)$ 个点 (P_k) 作为“候选边界点”，若 (P_k) 在当前圆外，则以 (P_i, P_j, P_k) 为三点计算外接圆（此时圆覆盖 (P_i, P_j, P_k) ）。



感谢同学们认真听课!

欢迎同学们积极提问、交流

mcstj@mail.sysu.edu.cn